5

Virtual Machines

5.1 Introduction

The previous chapter describes the basic infrastructure of a data center, including physical infrastructure, such as power and cooling, as well as racks, pods, servers, networks, and storage facilities. This chapter begins the description of virtualization mechanisms and technologies that allow a cloud owner to use the facilities to provide elastic computing. The chapter explores the concept of a Virtual Machine and the support systems needed to create and manage Virtual Machines. The next chapters describe an alternative virtualization technology used in data centers, containers. Subsequent chapters consider virtual networks and virtual storage facilities.

5.2 Approaches To Virtualization

The concept of virtual machines existed long before cloud computing was invented. The technologies used to implement virtual machines can be divided into three broad categories:

- Software emulation
- Para-virtualization
- Full virtualization

Software emulation. The idea of using software to emulate a computer has many applications. One use involves accommodating heterogeneous computers. Suppose a user who has a particular type of computer, C_1, wants to run a program, *P*, that has been

compiled for another type of computer, C_2. A piece of software called a *software emulator* solves the problem. The emulator treats program P as data by repeatedly reading an instruction, doing whatever C_2 would do, and then moving to the next instruction.

The software emulation approach has been especially popular as a way to make a new programming language available quickly. Before a language can be used on a particular type of computer, someone must write a compiler that translates programs into instructions the computer understands. Writing a compiler is time-consuming and error prone, and often limits how quickly a new language can be widely adopted. Software emulation allows a language designer to write one compiler and then use the compiler on all types of computers.

To implement a compiler for a new programming language, use an existing language to build two tools: a compiler and a software emulator. To simplify writing the tools, define a hypothetical computer (i.e., a type of virtual machine). Build a compiler that translates programs written in the new language into code for the hypothetical machine. Build a software emulator that reads code for the hypothetical machine and carries out the steps the code specifies. The community uses the term *byte code* to describe instructions for the hypothetical computer, calls the compiler a *byte code compiler*, and calls the emulator a *byte code interpreter*. Once the compiler and software emulator have been written, porting them to a new type of computer is much easier than building a new compiler†.

Although it offers the advantages of being general and making it easy to move apps to new types of computers, software emulation has a major disadvantage that makes it unattractive as the primary form of virtualization used in data centers: software emulation adds significant overhead resulting in slow execution. The point is:

> *Although it increases the portability of compilers and other apps, software emulation incurs significant execution overhead, making it ineligible as a primary form of virtualization in a data center.*

Para-virtualization. Now classified as *para-virtualization*, an approach to virtualization pioneered in the 1960s is designed to allow multiple operating systems to run on a single computer at the same time. Of course, the virtualization mechanism cannot allow a given operating system to take over the underlying hardware completely. Instead, para-virtualization uses a piece of software known as a *hypervisor* to control multiple operating systems and move the processors among them analogous to the way a single operating system moves the processor among a set of running processes.

Unlike software emulation, para-virtualization allows software to run at high speed. That is, para-virtualization avoids the inherent overhead of software emulation and allows the processor to execute instructions directly with no extra software involved. We say that para-virtualization allows instructions to execute *natively*. A later section describes how native execution is possible.

To prohibit a given operating system from commandeering the hardware, para-virtualization requires the code for an operating system to be altered before it can be

†The Java Programming Language uses the software emulation approach, and defines a *Java Virtual Machine* (*JVM*).

used. All *privileged instructions* (i..e., instructions used to gain control of the hardware and I/O devices) must be replaced by calls to hypervisor functions. Thus, when an operating system tries to perform I/O, control passes to the hypervisor, which can choose whether to perform the requested operation on behalf of the operating system or deny the request.

> *An early form of virtualization known as* para-virtualization *allows multiple operating systems to run on a computer at the same time by using a piece of software known as a* hypervisor *to control the operating systems. Para-virtualization has the advantage of allowing high-speed execution and the disadvantage of requiring code to be altered to replace privileged instructions before it can be run.*

Full virtualization. Like para-virtualization, the approach known as *full virtualization* allows multiple operating systems to run on the same computer at the same time. Furthermore, full virtualization avoids the overhead of software emulation and allows operating system code to run unaltered. The next sections explain full virtualization in more detail. For now, it is sufficient to know:

> *Full virtualization allows multiple operating systems to run on a single computer at the same time and at high speed without requiring operating system code to be altered.*

5.3 Properties Of Full Virtualization

The full virtualization technologies currently used to support Virtual Machines (VMs) in cloud data centers have three key properties:

- Emulation of commercial instruction sets
- Isolated facilities and operation
- Efficient, low-overhead execution

Emulation of commercial instruction sets. To a customer, a VM appears to be identical to a conventional computer, including the complete instruction set. Code that has been compiled to run on a commercial computer will run on a VM unchanged. In fact, a VM can boot and run a commercial operating system, such as *Microsoft Windows* or *Linux*.

Isolated facilities and operation. Although multiple VMs on a given server can each run an operating system at the same time, the underlying system completely isolates each VM from the others. From the point of view of an operating system running on a VM, the operating system thinks it controls all of physical memory, a set of I/O devices, and the processor, including all cores.

Efficient, low-overhead execution. The most important and surprising aspect of VM technology arises from the low overhead a VM imposes. In fact, until one understands how a VM works, performance seems impossibly fast because most instructions, especially those used in application code execute natively (i.e., at full hardware speed). To summarize:

> *When an application runs on a VM, most instructions execute as fast as they do when the application executes directly on the underlying processor.*

The next sections explain the conceptual pieces of software that a VM system uses and how the implementation can achieve such high performance.

5.4 Conceptual Organization Of VM Systems

The general idea behind VM is straightforward: load software onto a server that allows the cloud provider to create one or more VMs. Allow the tenant who owns each VM to boot an operating system on the VM, and then use the operating system to launch one or more applications.

The key piece of software responsible for creating and managing VMs is known as a *hypervisor*. We think of a hypervisor as controlling the underlying hardware. Each VM the hypervisor creates is independent of other VMs. Figure 5.1 illustrates the conceptual organization of a server running hypervisor software, and a set of VMs that each run an operating system and apps.

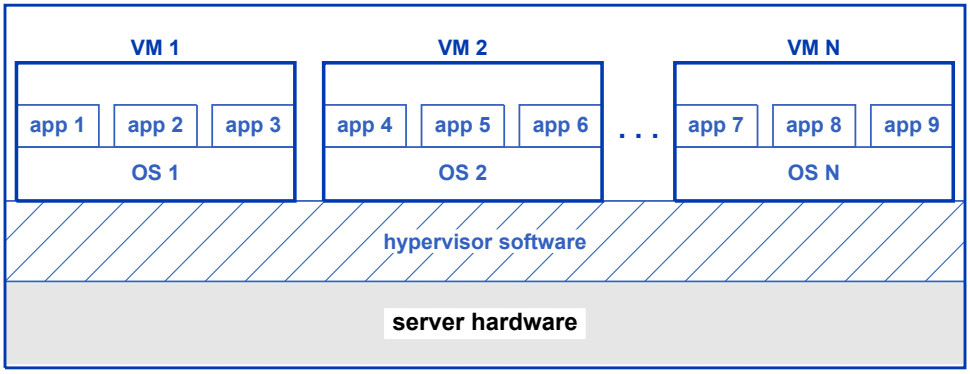


Figure 5.1 Illustration of a server running a hypervisor† and a set of VMs.

†Technically, the figure shows a *type 1 hypervisor*, the type commonly used in data centers; a later section describes a *type 2 hypervisor* that an individual might run on their computer.

5.5 Efficient Execution And Processor Privilege Levels

The question arises, how can an application running in a VM operate at hardware speed? Figure 5.1 makes it appear that two layers of software separate a running application from the underlying hardware: an operating system and a hypervisor. We know that software operates much slower than hardware, so how is high speed possible?

To understand how a VM runs apps at hardware speed, consider how an operating system runs apps on a conventional computer. When a user launches an app, the operating system loads the code for the app into the computer's memory. The operating system then instructs the processor to start executing the code. Execution proceeds at the hardware rate because the processor executes code for the app directly without going "through" the operating system.

An application cannot be allowed to execute all possible instructions or the computer would be vulnerable to hackers who might steal information or use the computer in a crime. To prevent such problems, the processor hardware used in a conventional computer has two *privilege levels* or *modes of operation*. Operating system code runs in *kernel mode*, which allows the operating system to perform all possible instructions. When it switches the processor to application code, the operating system also changes to *user mode*, which means only basic instructions are available. If the application makes a *system call* to request an operating system service (e.g., to read from a file), the processor transitions back to kernel mode. Figure 5.2 illustrates operating system and app code in memory and some of the possible transitions.

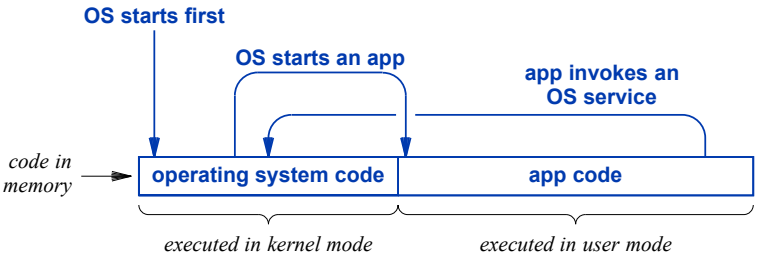


Figure 5.2 Illustration of operating system and app code in memory. The processor executes each at the same high speed, changing mode when transitioning from one to the other.

User mode means an app can access memory that has been allocated to the app, and the app can perform basic instructions, such as addition or subtraction. However, an app cannot access memory owned by the operating system or the memory owned by other apps. If an app attempts to make an instruction that requires privilege or attempts an illegal memory access, the processor raises an exception, which transfers back to a special point in the operating system to allow the operating system to handle the problem.

5.6 Extending Privilege To A Hypervisor

When a server runs a hypervisor and VMs, the software uses the same approach as an operating system uses to run apps, except that the processor employs three levels of privilege: one for the hypervisor, a second for an operating system, and a third for apps. Only the hypervisor can create a VM and allocate memory to the VM. The operating system is restricted to the memory that has been allocated to its VM. The operating system can run apps in the memory it has been allocated. As with a conventional computer, the processor always runs code directly from memory, allowing the code to be executed at hardware speed.

Figure 5.3 illustrates software in memory when a VM runs and shows some of the transitions among privilege levels as a hypervisor starts a VM, the operating system in the VM launches an app, the app calls an operating system service, and the operating system eventually exits (i.e., shuts down the VM).

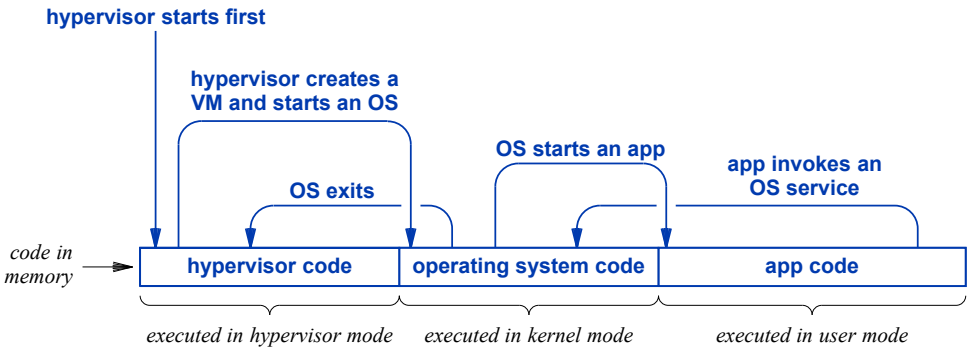


Figure 5.3 Illustration of transitions among code in memory when a hypervisor runs a VM with an operating system and the operating system runs an app.

5.7 Levels Of Trust

We can think of the three processor modes as imposing three levels of trust. When it runs in hypervisor mode, the processor can perform any operation available on the hardware. Thus, the hypervisor code is trusted completely. When it runs in kernel mode, the processor restricts the set of operations to ensure that the operating system cannot affect other VMs or the hypervisor. Because the hardware mode prevents interferences, the operating system code does not need to be trusted as much as hypervisor code. Finally, when it runs an application in user mode, the processor restricts the set of operations even further, making it impossible for an app to affect other apps or the operating system. Because the hardware prevents interference, an app does not need to be trusted as much as an operating system.

Of course, each entity in the trust hierarchy need to be trusted to manage entities under it. When it creates a VM, a hypervisor must be trusted to configure the hardware correctly to ensure the VM is isolated from other VMs. Similarly, when an operating system runs an app, the operating system must be trusted to configure the hardware to keep the app protected from other apps. If an entity attempts to perform an operation that exceeds its trust level, control passes up to the entity at the next trust level. We can summarize:

> *The software used for VMs runs a trust hierarchy controlled by processor modes. A hypervisor is trusted to establish and manage VMs, an operating system is trusted to create and manage a set of apps, and an app runs with the least trust. An attempt to exceed the trust boundary causes the processor to return control to the software at the next higher trust level.*

5.8 Levels Of Trust And I/O Devices

Adding an extra level of privilege makes I/O devices problematic. To understand why, consider how an operating system manages I/O devices (e.g., a screen, keyboard, disk, and network interface) on a conventional computer. When it boots on a conventional computer, the operating system uses a hardware mechanism known as a *bus* to communicate with I/O devices. The first step consists of sending a series of requests across the bus to form a list of all I/O devices that are present. The operating system must include *device driver* software for each device. The operating system uses the device driver code for a given device to control the device hardware and handle all communication with the device.

We can now understand why a dilemma arises when a server runs multiple virtual machines. Each virtual machine will run its own operating system, and a virtual machine can boot a standard operating system. On the one hand, a standard operating system will try to use the bus to take ownership of all I/O devices. On the other hand, the hypervisor cannot allow one of the operating systems to gain exclusive control of I/O devices because the devices must be usable by all virtual machines. In particular, because all VMs need network communication, no single VM can be allowed to commandeer the server's network I/O.

5.9 Virtual I/O Devices

VM technology uses an interesting approach to handle I/O: *virtual I/O devices*. When it creates a VM, a hypervisor creates a set of *virtual I/O devices* for the VM to use. A virtual I/O device is implemented by software. When an operating system on a VM attempts to use the bus to access an I/O device, the access violates privilege, which means the hypervisor is invoked. The hypervisor runs the appropriate virtual device

software, and then arranges to send the response to the operating system as if a physical device responded. That is, the hypervisor makes it appear that the response appears to come over the bus. The point is:

> *From an operating system's point of view, a virtual device is indistinguishable from a real device because communication with a virtual device uses the same bus operations that communication with a physical device uses.*

5.10 Virtual Device Details

The use of virtual devices introduces two approaches, and both have been used:

- Invent a new, imaginary device
- Emulate an existing hardware device

Invent a new, imaginary device. Because it is implemented with software, a virtual device can have any properties a programmer imagines†. Communication between the operating system and the device uses the standard bus operations, but the details of what I/O operations the device supports and how data moves between the operating system and the device are controlled by the virtual device, which gives an opportunity for an improved design. Most commercial device hardware uses a complicated, arcane interface that makes writing device driver software difficult. In principle, a programmer who creates a virtual device can create a beautiful, clean design that avoids the messy hardware details, making it much easier to write device driver software. It may also be possible to increase efficiency (e.g., by creating an imaginary disk with large blocks).

Emulate an existing hardware device. The alternative to creating an imaginary device consists of building software that emulates an existing hardware device. To understand the motivation for emulation, consider the availability of device driver software. When it boots and discovers a device, an operating system must load device driver software that can interact with the device (i.e., send I/O requests to the device and receive responses). Because the device driver software used with one operating system is incompatible with other operating systems, a separate driver must be created for each operating system. Writing and debugging device driver code is tedious, time-consuming, and requires expertise with a specific operating system. Building virtual device software that emulates an existing hardware device avoids having to build new device drivers — an operating system can load and run existing device driver software

The downside of building a virtual device that emulates an existing hardware device arises because the virtual device must behave exactly like a real hardware device. It must accept all the same requests and respond to each request exactly like the hardware device responds. Nevertheless, virtual machine systems have been deployed in which virtual devices emulate commercial hardware devices.

---

†For an example, see *virtIO* at URL https://docs.oasis-open.org/virtio/virtio/v1.0/virtio-v1.0.html.

5.11 An Example Virtual Device

As an example of a virtual device, consider a *virtual disk*†. Because data centers locate storage facilities separate from servers, all disk I/O requires communication over the data center network. Virtual disk software handles the situation easily by providing a standard disk interface to an operating system and also communicating over the network. Figure 5.4 illustrates the conceptual arrangement.

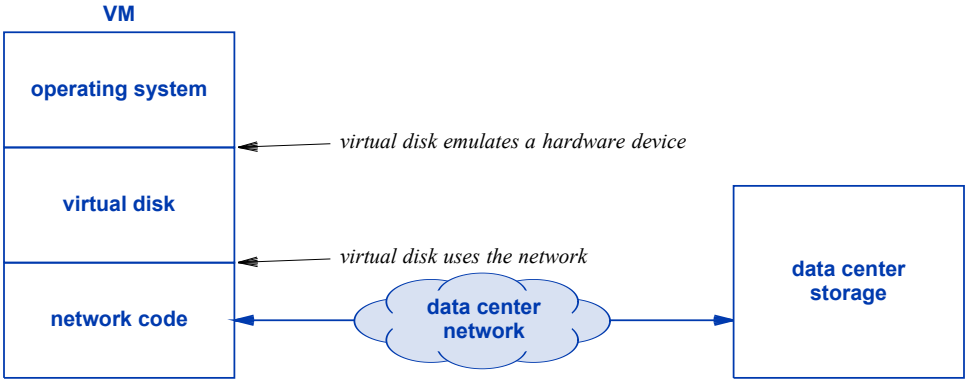


Figure 5.4 The conceptual organization of an operating system, virtual disk code, and network code.

As the figure shows, virtual disk code occupies a position between an operating system that runs in a VM and the network code in the hypervisor. When it communicates with the operating system, the virtual disk acts like a hardware device, using standard bus communication to receive requests and supply responses. For a disk, a request either writes data to the disk or reads data from the disk. Each time it receives a request, the virtual disk code uses the network to communicate with the storage facility in the data center. The virtual disk specifies the VM that made a request, and either sends data to be stored on the VM's disk or requests a copy of data from the VM's disk.

5.12 A VM As A Digital Object

A VM differs from a real server in a significant way: a VM is created and managed entirely by software. A hypervisor must keep a record of the VM, the region(s) of memory that have been allocated to the VM, the virtual I/O devices that have been created for the VM (including disk space that has been allocated in the data center storage facility), and the current status of the VM (e.g., whether the VM is currently running or has been suspended to allow another VM to run).

†Chapter 8 describes virtual disks and other data center storage mechanisms in more detail.

Having a complete record of each VM in a hypervisor has an interesting consequence: a VM can be turned into a digital object. That is, the entire VM can be transformed into a set of bytes. To understand how, imagine that a VM is currently suspended. The hypervisor has a record of all the memory the VM is using, so the VM's memory segments can be collected. Imagine, for example, that they are placed in a special file. The code and data for the operating system and apps that the VM is running are all stored in memory, so when the VM's memory has been collected, the OS and apps will be collected. Similarly, the virtual devices that have been created for the VM consist of software, so they can be collected as well. The key idea is:

> *Because a VM is implemented with software, all the pieces of a VM can be collected together into a digital object.*

5.13 VM Migration

The ability to collect all the pieces of a VM into a digital object has great significance, and data centers take advantage of it. Suppose, for example, that a data center manager receives an alert that a power supply is about to fail on a server. The manager can instruct the hypervisor to stop all the VMs running on the server, save a copy of each VM on disk, and then shut the server down. The manager can replace the server hardware, reboot the server, reload all the VMs, and resume operating, allowing each of them to continue where it left off.

Although it allows one to stop and restart a server, the ability to convert a VM to a digital object enables something much more important in cloud data centers: *VM migration*. That is, a VM can be moved. The basic idea is straightforward: stop a VM that is running on one server, convert the VM to a digital object, send the bytes across the network to a new server, and resume the VM on the new server. That is, a hypervisor on one server converts a VM into a digital object, sends the object to the hypervisor on another server, and the receiving hypervisor resumes the VM on its server. Although the VM is moved and the hypervisors do the work, the cloud industry says the VM *migrates* from one server to another.

The ability to migrate VMs allows a provider to rebalance loads dynamically to eliminate hot spots. Suppose, for example, that the VMs running on server X all perform intensive long-running computations with little or no I/O activity. Further suppose that the VMs running on server Y transfer huge amounts of data, but do not spend much time computing. The processor on server X will be saturated while the processor on server Y will remain relatively idle. Meanwhile, the I/O bus on server Y will be saturated while the bus on X is relatively idle. Migration allows a data center owner to balance the two servers by moving some of the VMs from Y to X and some of the VMs originally on X to Y. As a result, neither server will have a saturated processor or a saturated I/O bus.

Migration has many other uses. For example, during periods of especially light load, a data center manager can migrate VMs away from an area of servers and then power down the empty servers to reduce power consumption. A set of VMs owned by a given customer can be migrated to the same pod (or adjacent pods), reducing the network traffic that must cross the data center and reducing the latency among the customer's VMs.

5.14 Live Migration Using Three Phases

We have seen that it is possible to convert a VM into a digital object if the VM has been stopped. Unfortunately, it takes time to stop a VM, send it across a network, and then restart it. What happens if the VM is communicating over the network? Perhaps the VM is accessing a database or downloading a file. In such cases, stopping a VM can cause the other side of the communication to give up or declare the VM to have aborted. Instead of stopping a VM, engineers have devised a migration scheme that allows *live migration*. That is, a VM continues to run while migration proceeds.

To enable live migration, the amount of time a VM is unavailable must be minimized. The technique used divides migration into three phases:

- Phase 1: pre-copy
- Phase 2: stop-and-copy
- Phase 3: post-copy

Phase 1: pre-copy. In phase 1, the entire memory of the VM is copied to the new server while the VM continues to run. Of course, some pages in memory will be changed after they are copied; a record is kept of the pages that changed.

Phase 2: stop-and-copy. In phase 2, the VM is temporarily suspended, and any pages that changed after the phase 1 copy are copied again. The virtual memory system on modern servers makes it easy to detect which pages have been changed (i.e., are *dirty*). Thus, the set of changed pages can be identified quickly.

Phase 3: post-copy. In phase 3, the hypervisor sends remaining state information to the hypervisor on the new server. The state information includes items that are needed to resume (i.e., unsuspend) the VM, including the contents of registers when the VM was suspended. The hypervisor on the new server uses the information to allow the VM to continue executing.

Although a VM must be suspended temporarily, pre-copying the entire memory means almost no time is required during the second and third phases. To summarize:

> *A three-phase approach that minimizes the time a VM is unavailable makes live migration feasible.*

5.15 Running Virtual Machines In An Application

The previous sections describe how VM technology uses a hypervisor that runs directly on server hardware. Interestingly, an alternative form of VM technology has been developed that allows a hypervisor to run on a conventional operating system (i.e., on a user's laptop). A user launches a *hypervisor application* called a *hosted hypervisor*†. A hosted hypervisor runs along with other applications. Once it starts, the hosted hypervisor allows a user to create and manipulate VMs.

On a conventional computer, we think of an operating system rather than a hypervisor as owning and controlling the hardware. We use the term *host operating system* to describe the main operating system. When the user launches an application, the application runs as an independent process. Similarly, when the user launches a hosted hypervisor, the hypervisor runs as an application process. We say that the hosted hypervisor runs in *user space*.

As expected, each VM must run an operating system, which is called a *guest operating system*. A guest operating system does not need to be the same as a host operating system, and guest operating systems may differ from one another. For example, a user with a Mac laptop can use a hosted hypervisor to create a VM that runs Linux and another VM that runs Windows. Figure 5.5 shows the arrangement of software components after a user has launched a hosted hypervisor that runs two VMs along with four other applications.

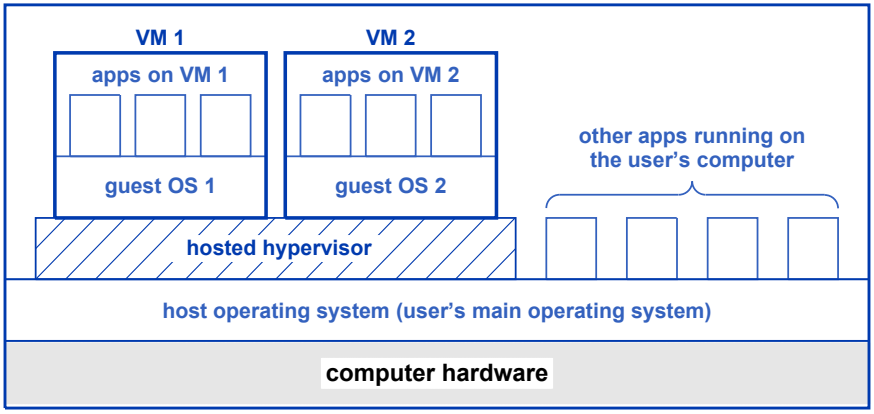


Figure 5.5 The conceptual arrangement of software on a conventional computer that results when a user runs a hypervisor application and creates two VMs.

Three questions arise concerning a system that runs a hosted hypervisor on a conventional computer.

---

†A hosted hypervisor is also classified as a *type 2* hypervisor to distinguish it from a *type 1 hypervisor* that runs directly on server hardware.

- How is it possible?
- What benefit does it offer a user?
- Is the technology useful in a cloud data center?

Once we understand how a hosted hypervisor works, we will discuss how a user can benefit. The next chapter answers the question about whether the technology is useful in a cloud data center by describing one possible use: running a guest operating system that can run containers.

5.16 Facilities That Make A Hosted Hypervisor Possible

Two facilities allow a hosted hypervisor and a guest OS to operate correctly without high privilege: a processor mechanism that supports virtualization and a way to map guest OS operations onto the host operating system's services. Modern processors include a special virtualization mechanism that enables a hosted hypervisor to create VMs that each run a guest OS. Although the mechanism involves many complex details and subtleties, the overall result is straightforward: the mechanism allows a guest OS to proceed as if it were running at the highest privilege level, even though it does not have privilege and does not have complete access to the underlying hardware. For example, although it is only given access to a portion of the physical memory, a guest OS remains unaware that it has a limited view.

A hosted hypervisor arranges a mapping between I/O requests that a guest OS makes and services in the host operating system. For example, when a guest OS communicates over a network, the hosted hypervisor forwards the requests to the host operating system in such a way that the guest OS has the illusion of using a hardware interface. The hypervisor may choose to assign each guest OS a unique network address, making it appear that each guest OS has its own network interface. Similarly, the hypervisor forwards disk I/O requests to the host operating system in such a way that it gives the guest OS the illusion of a separate disk.

Interestingly, a hosted hypervisor can arrange for some files or file systems to be shared between the guest and host operating systems. Shared files provide an efficient way for applications running in a VM to share data with applications running on the host. The point is:

> *Because a hosted hypervisor can map I/O requests from a guest operating system onto services offered by the host operating system, it is possible for applications running on VMs and applications running on the host to share files.*

5.17 How A User Benefits From A Hosted Hypervisor

A hosted hypervisor allows a user to run multiple operating systems on a single computer. Multiboot mechanisms also allow a user to boot multiple operating systems, but a user can only boot one system at a given time. A hosted hypervisor has a distinct advantage because a user can run multiple systems simultaneously and can switch from one system to another without rebooting the computer.

To understand how such a system can be useful, suppose a user who runs Mac OS on their laptop wishes to use an application that only runs on Linux. With a hosted hypervisor, the user can launch a hosted hypervisor, boot Linux in a window (or virtual desktop), and then use Linux to run the desired application. Furthermore, the user can leave the virtual machine running, and switch back and forth between Linux and Mac OS simply by switching windows or desktops. The point is:

> *Unlike a multiboot mechanism, a hosted hypervisor allows a user to run multiple operating systems simultaneously and switch among them quickly.*

5.18 Summary

Cloud data centers make extensive use of Virtual Machine (VM) technology. Software known as a hypervisor allows a user to create and manage VMs. Conceptually, the hypervisor owns the underlying hardware, VMs run over the hypervisor, and each VM runs an operating system plus apps. In practice, the code for a hypervisor, operating system, and apps reside in memory, and the hardware executes code directly until the code attempts to perform an operation that requires higher privilege. If an app attempts to exceed its privilege level, control passes to the operating system that started the app. If an operating system attempts to exceed its privilege level, control passes to the hypervisor. We say that the levels of privilege impose a hierarchy of trust.

A hypervisor provides each operating system with the illusion that it has direct access to peripheral devices over a conventional I/O bus. The mapping allows for arbitrary action, such as sending requests over a network to a remote storage facility when an operating system performs disk I/O.

All pieces of a VM can be collected into a digital object, which can be sent over a network, allowing hypervisors to migrate a VM from one physical server to another. A three-phase migration procedure minimizes the time during which a VM is unavailable, making live migration feasible. Data center owners use migration to rebalance the load on servers.

An alternative form of VM technology allows a user to run a hosted hypervisor on a conventional computer. Doing so means a user can switch among operating systems without rebooting the computer.